

令和5年度 修士論文

Deep Q-Network による列車制御方策の学習

Learning of train control strategies

by means of Deep Q-Network

研究指導教員 教授 高橋 聖

日本大学大学院理工学研究科

博士前期課程情報科学専攻

M2002 五十嵐 祥吾

Deep Q-Network による列車制御方策の学習

Learning of train control strategies

by means of Deep Q-Network

日本大学大学院理工学研究科博士前期課程情報科学専攻

高橋研究室

No.2002 五十嵐祥吾

概 要

近年、鉄道をはじめとする運輸業界では、少子化による人手不足が問題となっており、特に地方では都市への人口集中が合わさり深刻である。そのため、自動列車運転（Automatic Train Operation: ATO）システムによる、列車の自動運転化が急務となっている。ATO システムにおける列車制御方策としては、PID 制御や予見 Fuzzy 制御による手法が実用化されている。しかし、予め生成した運転曲線に基づき制御を行うため柔軟性に劣る。予見 Fuzzy 制御による手法は、全走行区間の制限速度、勾配を考慮した制御はモデルが複雑となり困難である。

本研究では、繰り返しシミュレーションを行い、その結果から制御方策を学習する Deep Q-Network による、列車制御方策の学習手法を提案する。DQN は、シミュレーションの結果からニューラルネットワーク（NN）により制御方策を学習するため、全走行区間の制限速度、勾配を考慮した制御を期待できる。

キーワード

自動列車運転 列車制御 機械学習 強化学習 Deep Q-Network

目次

第 1 章	序論	4
1.1	研究目的	4
1.2	本論文の構成	5
第 2 章	Deep Q-Network	7
2.1	強化学習	7
2.2	Deep Q-Network	8
2.3	Prioritized Experience Replay	9
2.4	Ape-X	9
第 3 章	理想環境における列車制御方策の学習	11
3.1	章概要	11
3.2	提案手法	11
3.2.1	列車運行シミュレータ	12
3.2.2	報酬関数	13
3.2.3	遷移情報メモリ	15
3.2.4	学習器	15
3.2.5	推論器	17
3.2.6	変動ステップ周期	17
3.2.7	割引率の補正	18
3.3	評価方法	18
3.4	結果	20
3.5	考察	23
第 4 章	列車総重量が走行毎に異なる環境における列車制御方策の学習 ..	25
4.1	章概要	25
4.2	提案手法	25
4.2.1	列車運行シミュレータ	25
4.2.2	報酬	26
4.2.3	ネットワーク	26

4.2.4	学習環境	27
4.3	評価方法	28
4.4	結果	28
4.5	考察	29
第 5 章	前方列車が存在する環境における列車制御方策の学習	31
5.1	章概要	31
5.2	提案手法	31
5.2.1	報酬	31
5.2.2	ネットワーク	32
5.2.3	学習環境	34
5.3	評価方法	34
5.4	結果	35
5.5	考察	38
第 6 章	結論	40
	謝辞	41
	参考文献	42

第1章 序論

1.1 研究目的

近年、鉄道業界では少子化による人手不足が問題となっており、特に地方では都市への人口集中が合わさり、運転士の不足を要因とした減便も発生している[1]。また、図 1-1 に示す JR 四国における主な路線の運輸密度の推移から見て取れるように、地方の人口減少により地方鉄道の輸送密度が減少しており、鉄道会社の経営を圧迫している。

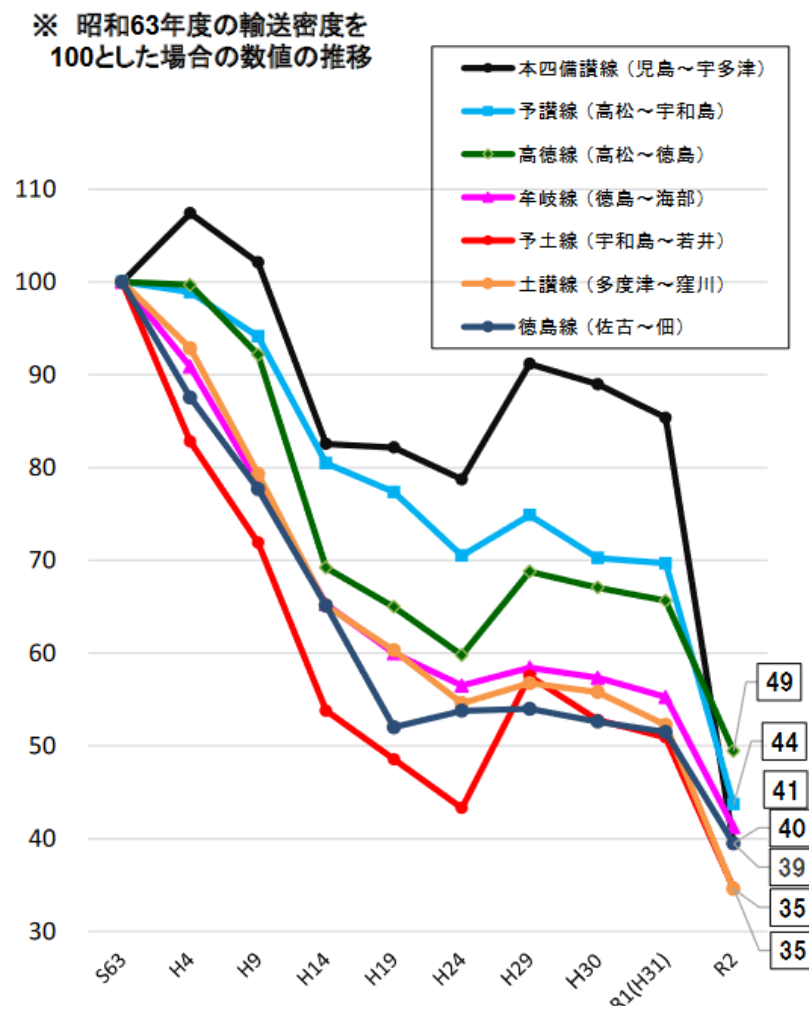


図 1-1 JR 四国における運輸密度の推移

出典: 国土交通省 ローカル鉄道を取り巻く現状

以上の様な鉄道業界の問題を解決するため、自動列車運転（Automatic Train Operation: ATO）システムによる列車運行の省人化が急務となっている。ATO システムにおける列車制御方策としては、PID 制御や予見 Fuzzy 制御による手法が実用化されている。PID 制御による ATO は、予め生成した運転曲線に基づき目標速度に追従する様 PID 制御を用いてノッチを決定する。したがって、勾配や曲線の多い路線、乗客数の変化により列車総重量が大きく変化する路線、過密なダイヤにより前方列車の遅延が制御に影響を与える路線では、設定した実際の運転が設定した運転曲線から乖離するため、定時性、省エネルギー性、乗り心地を全て実現することは困難である。予見 Fuzzy 制御による ATO は、定時性、省エネルギー性、乗り心地を考慮したメンバシップ関数を用いて制御を行うため、走行時間に余裕がある場合に惰行し、消費エネルギーを抑えるなど PID 制御による ATO と比較し柔軟な制御を行うことができる。しかし、全走行区間の制限速度、勾配、曲線を考慮した制御はメンバシップ関数が複雑となるため困難である。

そこで本研究では、画駅間走行時間、列車総重量が走行毎に異なる環境、前方列車の停車により駅間での停車を要する環境など様々な環境において、安全性、定時運転、駅停車位置精度、省エネルギー性、乗り心地の面で良好な制御を行う列車制御方策を Deep Q-Network により学習手法を提案する。DQN は、シミュレーションの結果からニューラルネットワーク（NN）により制御方策を学習するため、全走行区間の制限速度、勾配を考慮した制御を期待できる。

1.2 本論文の構成

第 2 章では Deep Q-Network、及び関連研究について述べる。第 1 節では、強化学習に関して基礎的な説明を述べる。第 2 節では、Deep Q-Network の元となった強化学習手法である Q 学習について述べる。第 3 節では、本研究で用いる強化学習である Deep Q-Network について述べる。第 4, 5 節では、Deep Q-Network において効率的に学習を行う手法である Prioritized Experience Replay, Double Q-Network について述べる。

第 3 章では、Deep Q-Network により他に列車が存在しない環境において列車

制御方策を学習する手法を提案し，提案手法を用いて学習した列車制御方策による制御が実際の運転手による制御と同様のものか確認する．

第 4, 5, 6 章ではそれぞれ，計画駅間走行時間が駅間走行毎に異なる環境，列車総重量が駅間走行毎に異なる環境，前方列車が存在し前方列車が駅間途中で一時的に停車する環境における列車制御方策の学習手法を提案し，提案手法を用いて学習した列車制御方策が安全性，定時性，駅停車位置精度，乗り心地，省エネルギー性の面で良好な制御を行うか確認する．

第2章 Deep Q-Network

2.1 強化学習

強化学習とは、ある環境の中で行動する主体（エージェント）の最適な行動方を適応的に学習する機械学習の1手法である [2].

図 2.1 に環境とエージェントの関係を示す. ここで環境の状態遷移はマルコフ決定過程 (Markov Decision Process: MDP) に従うものとする. すなわち, 状態 s_t における行動 a_t による状態 s_{t+1} への遷移確率 $P(s_{t+1}|s_t, a_t)$ は過去の状態に依存せず, 現在の状態 s_t 及び行動 a_t にのみ依存する. エージェントは環境から時刻 t における状態 s_t を受け取り方策関数 $\pi(s_t)$ に基づき行動 a_t を決定し環境に渡す. 環境はマルコフ決定過程に基づき時刻 $t+1$ における状態 s_{t+1} を, 報酬関数 $r(s_t, a_t, s_{t+1})$ に基づき報酬 r_{t+1} を決定しエージェントに渡す. 以上のステップを繰り返し, エージェントは報酬を得る.

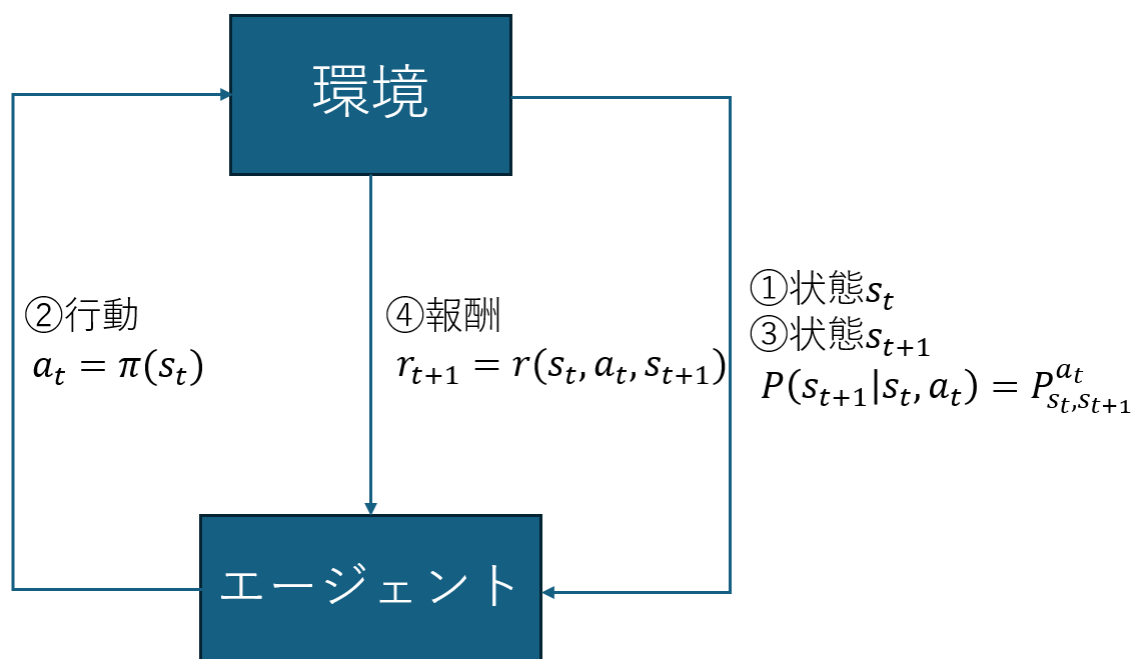


図 2.1 環境とエージェントの関係

強化学習は式(2.1)に示す現在時刻 t から未来までに得られる報酬の加重総和 v_t を最大化する方策関数 $\pi(s_t)$ を学習する

$$v_t = \sum_{i=t}^{\infty} \gamma^{i-t} r_i \quad (2.1)$$

ここで、 r_t は時刻 t における報酬、 γ は割引率($0 \leq \gamma \leq 1$)である。割引率 γ は未来に得られる報酬の重みを表し、この値が大きいほど未来に得られる報酬を重視する。強化学習は探索を行うことで、どの状態でどの行動を行うとどの程度の報酬を得られるかを取得し、これを用いて方策関数 $\pi(s_t)$ を学習する。

2.2 Deep Q-Network

Deep Q-Network (DQN) は、正解例が直接与えられない問題において、最適な行動選択法の適応的発見を目指す強化学習の手法である [3]。本手法は、行動主体（エージェント）がある状態 s において行動 a を取る価値を次式に示す $Q(s, a)$ として定義する。

$$Q(s_t, a_t) = r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) \quad (2.2)$$

ここで、 t は時刻、 r_{t+1} は状態 s_{t+1} に遷移した際の報酬であり、状態 s_{t+1} 、報酬 r_{t+1} は状態 s_t において実際に行動 a_t を行うことで求められる。 γ は報酬の時間的重みを表す割引率と呼ばれる値であり、 $0.9 \sim 0.999$ 程度の 1 に近い値が用いられる。式(1)に示されるように、 Q 値は行動 a により得られる即時的な報酬に加え、最適な行動を取り続けた場合に得られる報酬の加重総和から得られる。行動 a_t は ϵ -greedy 法により決定され、確率 ϵ で一様分布に基づきランダムな行動を、確率 $(1 - \epsilon)$ で $\arg\max_a Q(s_t, a)$ を取る。

DQN は、 $Q(s, a)$ を状態 s を入力し各行動 a_i の Q 値 $Q(s, a_i)$ を出力する NN を用いて近似する。DQN において、出力値が式(2.2)により求められる Q 値に近づくように NN は学習される。したがって、出力層の誤差関数 $E(s_t, a_t)$ は次式により与えられる。

$$E(s_t, a_t) = (r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))^2 \quad (2.3)$$

2.3 Prioritized Experience Replay

Prioritized Experience Replay (PER) は DQN において重要性の高い遷移情報を優先的にサンプリングする手法であり、次式によりサンプリングされる確率を決定する [4].

$$P(i) = \frac{(|\delta_i| + \epsilon)^\alpha}{\sum_k (|\delta_k| + \epsilon)^\alpha} \quad (2.4)$$

ここで、 ϵ は微小定数、 α は重要性による優先度を調整するパラメータであり、 $0 \leq \alpha \leq 1$ をとる。 δ は TD 誤差と呼ばれる値であり、次式により求められる。

$$\delta = r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \quad (2.5)$$

2.4 Ape-X

Ape-X は、PER を分散並列学習で行う手法である [5]。本手法は、図 2.2 に示す Actor, Replay, Learner の 3 つのモジュールで構成される。Actor は、1 つの CPU が割り当てられ、遷移情報の収集と初期優先度の算出を行う。Replay は、各 Actor から遷移情報を受け取り、蓄積する。また、PER によりミニバッチを作成し、Learner に受け渡す。Learner は、1 つの GPU が割り当てられ、Replay から受け取ったミニバッチを学習する。また、定期的にネットワークのパラメータを各 Actor に送信し、ネットワークの同期を行う。

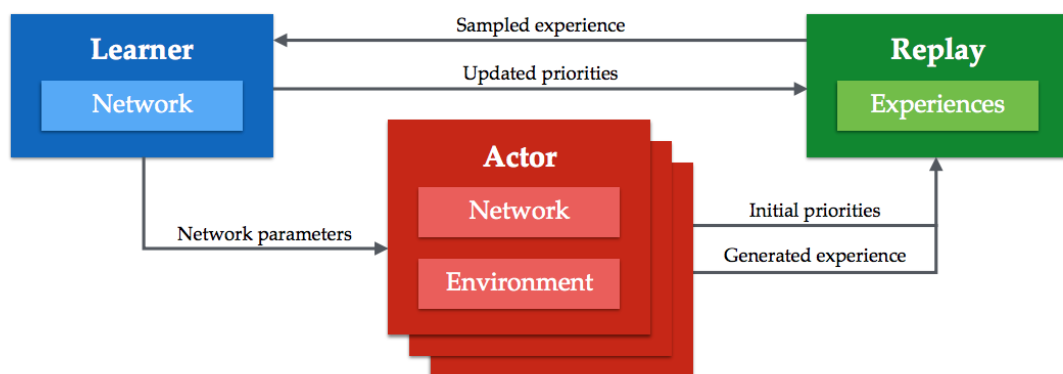


図 2.2 Ape-X

出典: Distributed Prioritized Experience Replay [5]

第3章 理想環境における列車制御方策の学習

3.1 章概要

本章では，走行毎に環境が変化せず，他に列車が存在しない理想環境における DQN による列車制御方策の学習手法を提案し，学習した制御方策による制御が実際の運転手による制御と同様であるか確認する．なお，本研究で使用する列車走行モデルは実際の列車の挙動を完全にシミュレートするものではないため，制御の特徴のみに注目してその類似性を評価する．

3.2 提案手法

図 3.1 に提案手法の概要図を示す．提案手法は **Ape-X** をベースに列車運転シミュレータ，推論器，報酬関数，学習器，遷移情報メモリから構成される．

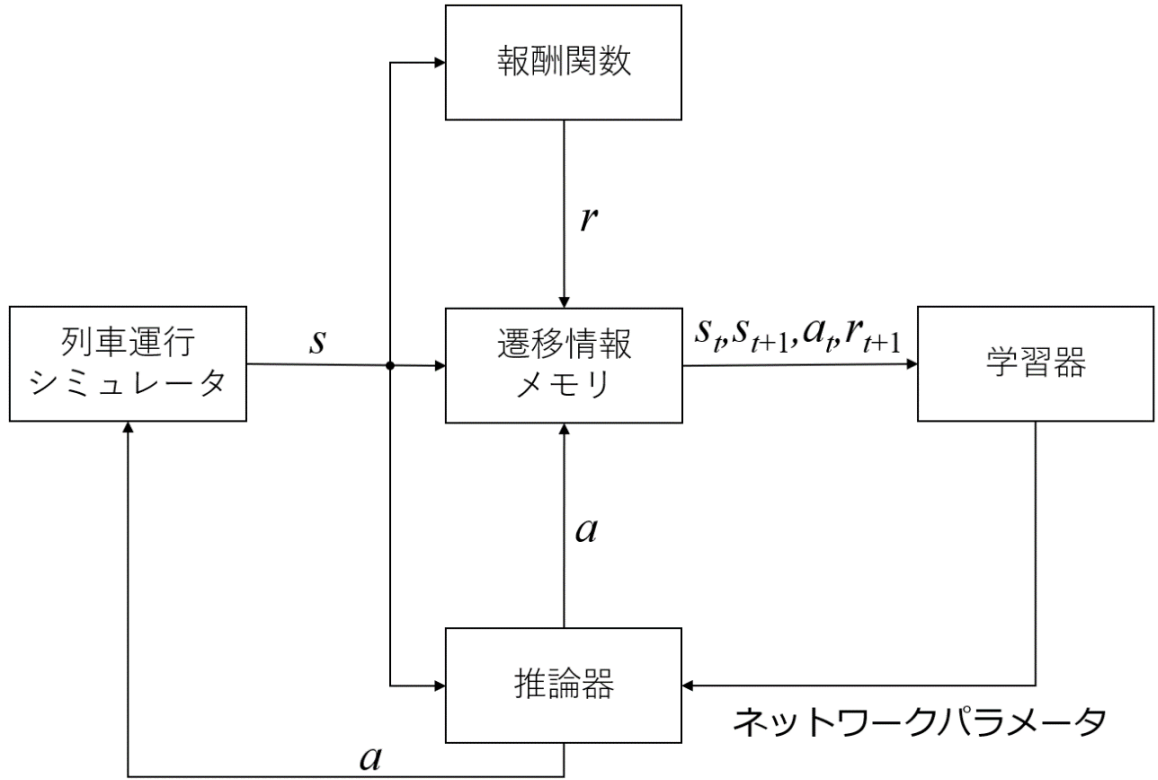


図 3.1 提案手法概要図

3.2.1 列車運行シミュレータ

列車運行シミュレータは制御入力（力行，惰行，制動）を受けとり，式(3.1)～(3.6)に示す列車走行モデルに従い物理演算を行い，列車位置，駅停車位置までの猶予距離，列車速度駅間走行猶予時間，制御保持時間，直前の制御入力から構成される状態 s_{t+1} を返す．

$$\frac{dx}{dt} = v \quad (3.1)$$

$$\frac{dv}{dt} = \begin{cases} -\frac{R_r + R_g + R_c}{28.34467} & \text{if } u = 0 \\ -8.64 - \frac{R_r + R_g + R_c}{28.34467} & \text{if } u = 1 \\ \frac{F - (R_r + R_g + R_c)}{28.34467} & \text{if } u = 2 \end{cases} \quad (3.2)$$

$$F = \begin{cases} -1.489v + 92.408 & \text{if } v < 42 \\ -0.4v + 46.68 & \text{if } 42 \leq v < 68 \\ -0.0963v + 26.0284 & \text{if } 68 \leq v \end{cases} \quad (3.3)$$

$$R_r = 2.39 + 0.0224v + 0.00062v^2 \quad (3.4)$$

$$R_g = g \quad (3.5)$$

$$R_c = \frac{800}{r} \quad (3.6)$$

ここで、 x は列車位置[km]、 v は列車速度[km/s]、 u は制御入力で $u=0$ で惰行、 $u=1$ で制動、 $u=2$ で力行、 F は原動機による引張力[kg/t]、 R_r は走行抵抗[kg/t]、 R_g は勾配抵抗[kg/t]、 R_c は曲線抵抗[kg/t]、 g は勾配[%]、 r は曲率半径[m]とする。

3.2.2 報酬関数

報酬関数は、状態と行動を受けとり、報酬関数に基づき報酬を返す。DQNにおいて、報酬はエージェントの状態を評価する値である。列車制御方策には、安全性、定時運転、駅停車位置精度、省エネルギー性、乗り心地が求められる。本研究では、これらの評価項目を定量的に評価し、次式で算出される報酬を与えた。

$$r = k_{sl}r_{sl} + k_{de}r_{de} + k_{te}r_{te} + k_{rc}r_{rc} + k_{ee}r_{ee} \quad (3.7)$$

ここで、 k_{sl} 、 k_{de} 、 k_{te} 、 k_{rc} 、 k_{ee} は報酬調整定数を表し、 $k_{sl}=1$ 、 $k_{de}=0.04$ 、 $k_{te}=7$ 、 $k_{rc}=5$ 、 $k_{ee}=0.02$ である。

r_{sl} は、式(3.8)により求められる安全性を評価する報酬であり、列車速度が制限速度を超過した場合に負の値が与えられる。ここで、 v_l は列車位置における制限速度[km/s]を表す。

$$r_{sl} = \begin{cases} -1\Delta t & \text{if } v_l < v \\ 0 & \text{otherwise} \end{cases} \quad (3.8)$$

r_{de} は、式(3.9)により求められる駅停車位置精度を評価する報酬であり、停止位置誤差が小さいほど高い値が与えられる。ここで x_s は規定停車位置[km]を表す。

$$r_{de} = \begin{cases} (\max(|x_s - x|, 10^{-4}))^{-1} & \text{if } x_s \leq x \wedge v = 0 \\ 0 & \text{otherwise} \end{cases} \quad (3.9)$$

r_{te} は、式(3.10)により求められる定時運転を評価する報酬であり、走行時間 t [s] が計画駅間走行時間 t_r [s]を超えた場合に、列車が規定停車位置 x_s から離れているほど低い値が与えられる。

$$r_{te} = \begin{cases} -(|x_s - x|)^{\frac{1}{3}} \Delta t & \text{if } t_r < t \\ 0 & \text{otherwise} \end{cases} \quad (3.10)$$

r_{rc} は、式(3.11)により求められる乗り心地を評価する報酬であり、頻繁に制御を切り替えた場合に負の値が与えられる。ここで、 t_h は制御入力保持時間[s], TH_{rc} は制御入力保持時間閾値[s]を表し、 $TH_{rc} = 10$ である。

$$r_{rc} = \begin{cases} \frac{t_h}{TH_{rc}} - 1 & \text{if } u_{t-1} \neq u_t \wedge t_h < TH_{rc} \\ 0 & \text{otherwise} \end{cases} \quad (3.11)$$

r_{ee} は、式(3.12)により求められる省エネルギー性を評価する報酬であり、消費エネルギーが大きいほど低い値が与えられる。消費エネルギーを求める方法は複数提案されているが、報酬を与えるにあたり J 単位の絶対的な消費エネルギーは必要ないため、本稿では単に加速度と速度の積を指標として与えた。ここで $u=2$ は力行を表す。

$$r_{ee} = \begin{cases} -f(u, v) v \Delta t & \text{if } u = 2 \\ 0 & \text{otherwise} \end{cases} \quad (3.12)$$

3.2.3 遷移情報メモリ

遷移情報メモリは、遷移前後の状態、行動、報酬から構成される遷移情報を蓄積し、サンプリングして学習器に与える。ここで、サンプリングには **Prioritized Experience Replay** を使用する。

3.2.4 学習器

学習器は、遷移情報を受けとり Q 関数を学習する。この際、 Q 値の発散を防ぐため NN の勾配を TensorFlow の `clip_by_global_norm` により L2 ノルムを 10 以内にクリップする。

NN には Dueling Network [6]を、最適化手法には Adam を用いた。また、 Q 関数の学習において学習対象 NN と次状態における Q 値の演算に用いる NN は Double DQN [7]を用いて分離した。NN の構成図を図 3.2 に示す。

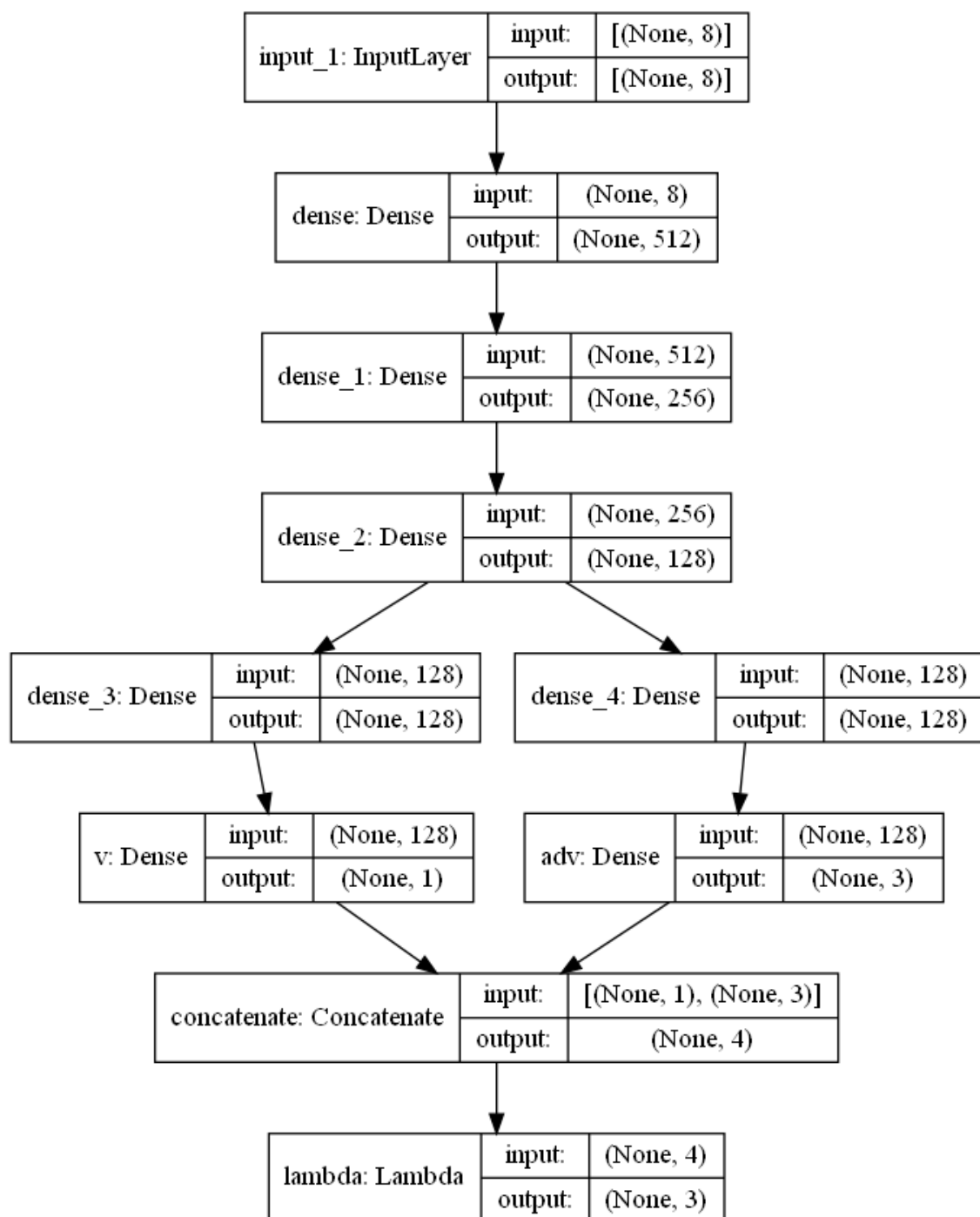


図 3.2 ネットワーク構成図

学習に用いた各パラメータを表 3.1 に示す.

表 3.1 学習器パラメータ

学習率	0.0001
バッチサイズ	512
Q 値割引率	0.9975
α (PER)	0.6
β 初期値(PER)	0.4
β 増加率(PER)[1 学習あたり]	0.00003
β 上限値(PER)	1
Double Q-Network 更新周期[学習回数]	512

3.2.5 推論器

推論器は、列車運行シミュレータから状態 s_t を受け取り、NN により各行動の Q 値を演算し、 ϵ -greedy 法により行動 a_t を選択、列車運行シミュレータに入力する．推論器は 50 機並列に動作し、各々が独立した NN を保持している．NN のパラメータは学習器の Double Q-Network 更新が更新される毎に学習器の NN と同期する．推論器 $i(i \geq 0)$ の ϵ -greedy 法におけるランダム行動率 ϵ_i は式(3.13)により求めた．

$$\epsilon_i = 0.001 + 0.08i \quad (3.13)$$

3.2.6 変動ステップ周期

一般的に DQN において、行動を決定する周期 (ステップ周期) は一定である．しかし、列車制御方策を学習するにあたり、駅停車位置精度のため駅停車前の減速には非常に精密な制御が求められる、ステップ周期を 0.1s 程度の短い時間に設定する必要があるが、常にステップ周期を 0.1s とした場合、シミュレーションに時間を要するため学習効率が悪い．そこで、通常時のステップ周期を 1s に、精密な制御が求められる停車駅 100m 手前からはステップ周期を 0.1s に設定し

た.

3.2.7 割引率の補正

ステップ周期を変動させる場合、割引率 γ が一定であると、ステップ周期が短い状態における報酬は長い状態における報酬と比べ余計に Q 値の割引が適用されるため過小評価される. 例として、割引率を 0.999 , $10s$ 後に報酬を得られるとしたとき現在の Q 値に対し $10s$ 後の報酬が反映される割合はステップ周期 $1s$ の場合 $\gamma^{10} = 0.99$, $0.1s$ の場合 $\gamma^{100} = 0.905$ となる.

基本となるステップ周期 Δt_{base} と任意のステップ周期 Δt において、ステップ周期に関わらず報酬を評価するために必要な割引率 γ_{fixed} の条件は式(3.14)により表せる.

$$\gamma_{fixed}^{\frac{1}{\Delta t}} = \gamma^{\frac{1}{\Delta t_{base}}} \quad (3.14)$$

すなわち式(3.15)を得る.

$$\gamma_{fixed} = \gamma^{\frac{\Delta t}{\Delta t_{base}}} \quad (3.15)$$

3.3 評価方法

DQN により学習した列車制御方策は NN のパラメータとして表現されるため直接的な評価は困難である. そこで、学習済みの列車制御方策を用いて列車運行シミュレータ上で駅 A～駅 B 間、及び駅 C～駅 D 間運転し、その結果から列車制御方策を評価した. また、学習した制御方策による制御と営業列車の制御を比較し、制御方策の妥当性を確認した. 駅 A～駅 B 間における勾配を図 3.3 に、駅 C～駅 D 間における勾配を図 3.4 に示す.

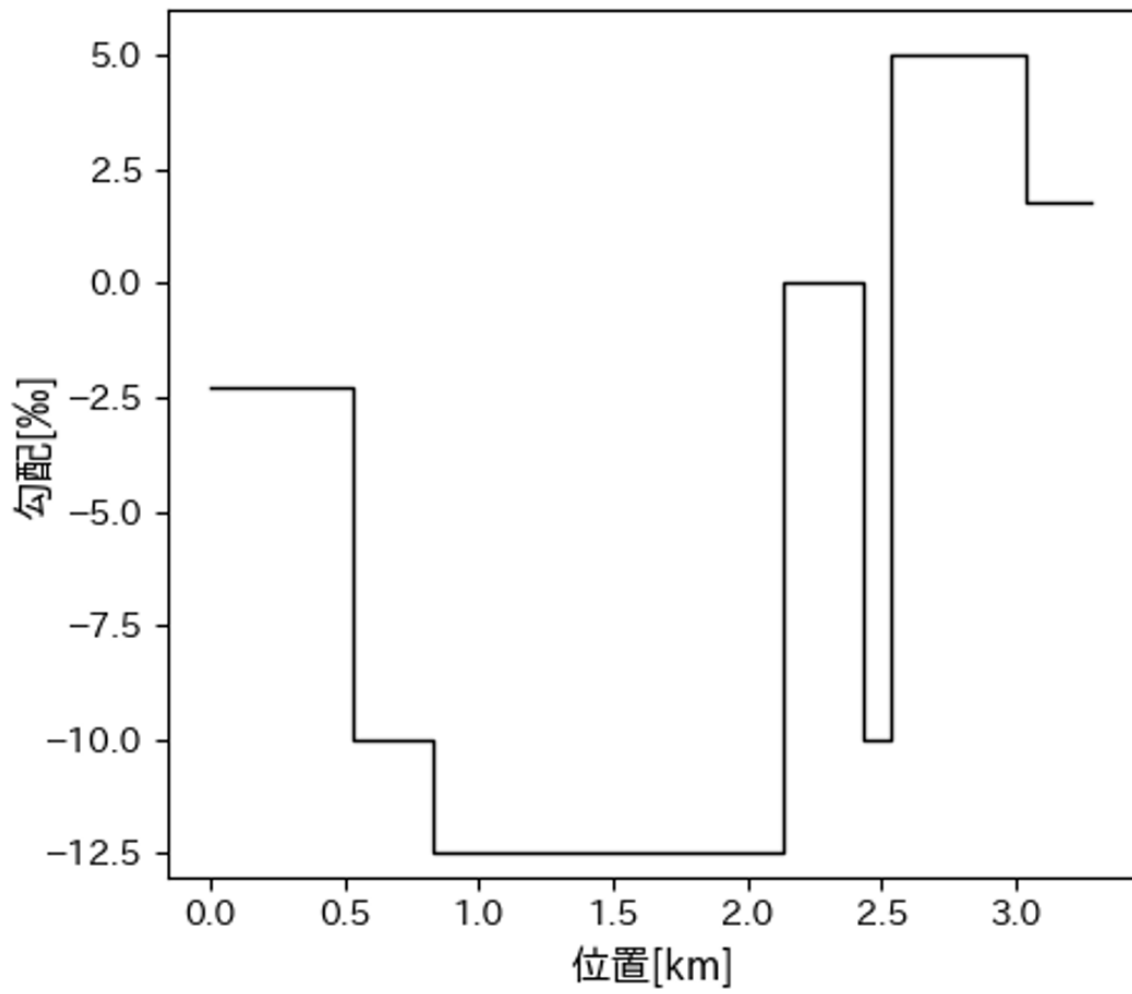


図 3.3 駅 A～駅 B における勾配

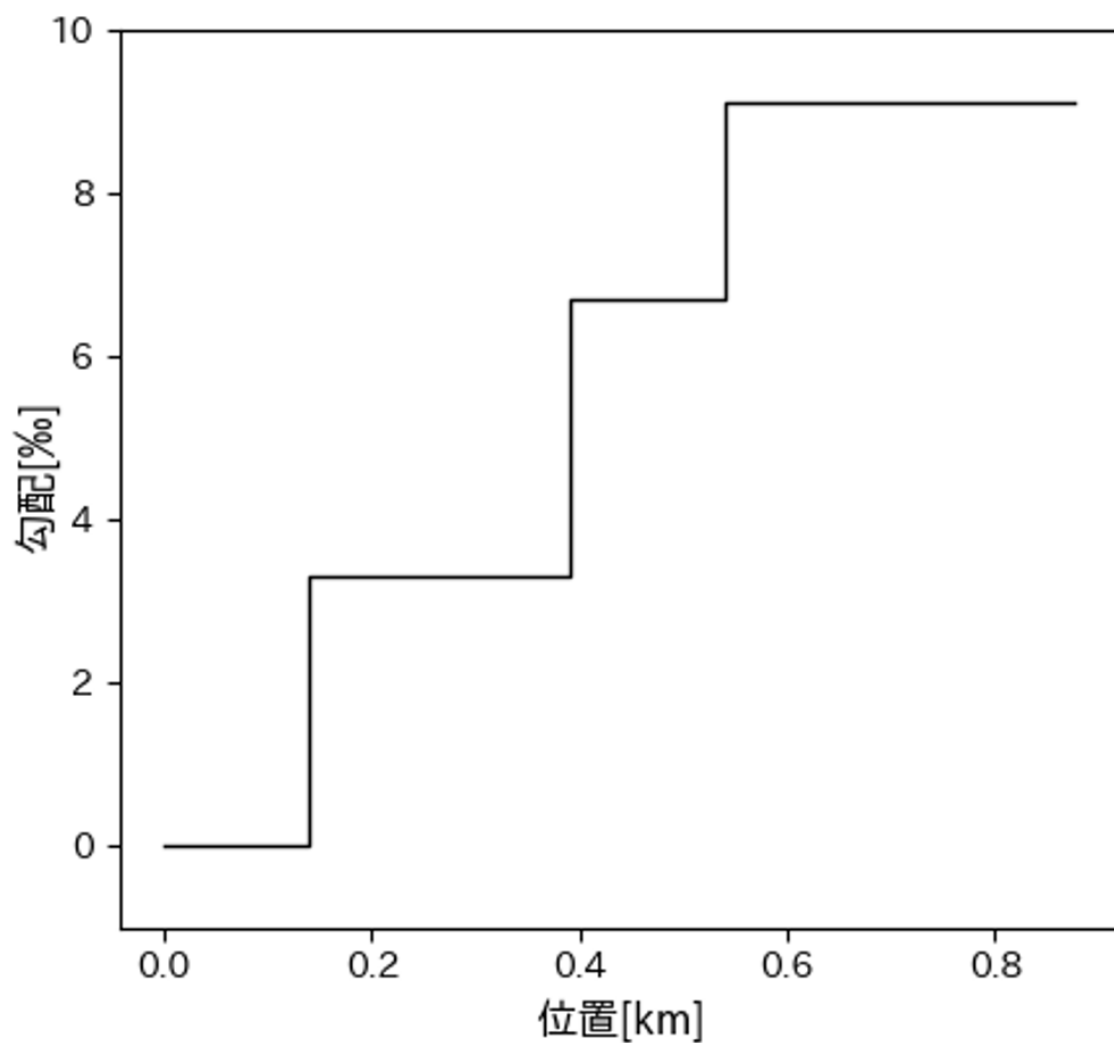


図 3.4 駅 C～駅 D における勾配

3.4 結果

列車運行シミュレータ上で学習したモデルにより A 駅～B 駅間, C 駅～D 駅間を走行することで得た制御入力, 運転曲線図, 同駅間における営業列車の運転曲線図を図 3.5, 図 3.6 に示す. ここで, 制御入力 0 は惰行, 1 は加速, 2 は減速を表す. また, 図 3.5, 図 3.6 に示した学習した制御方策による走行における駅間発生遅延時間, 駅停車位置誤差を表 3.2 に示す.

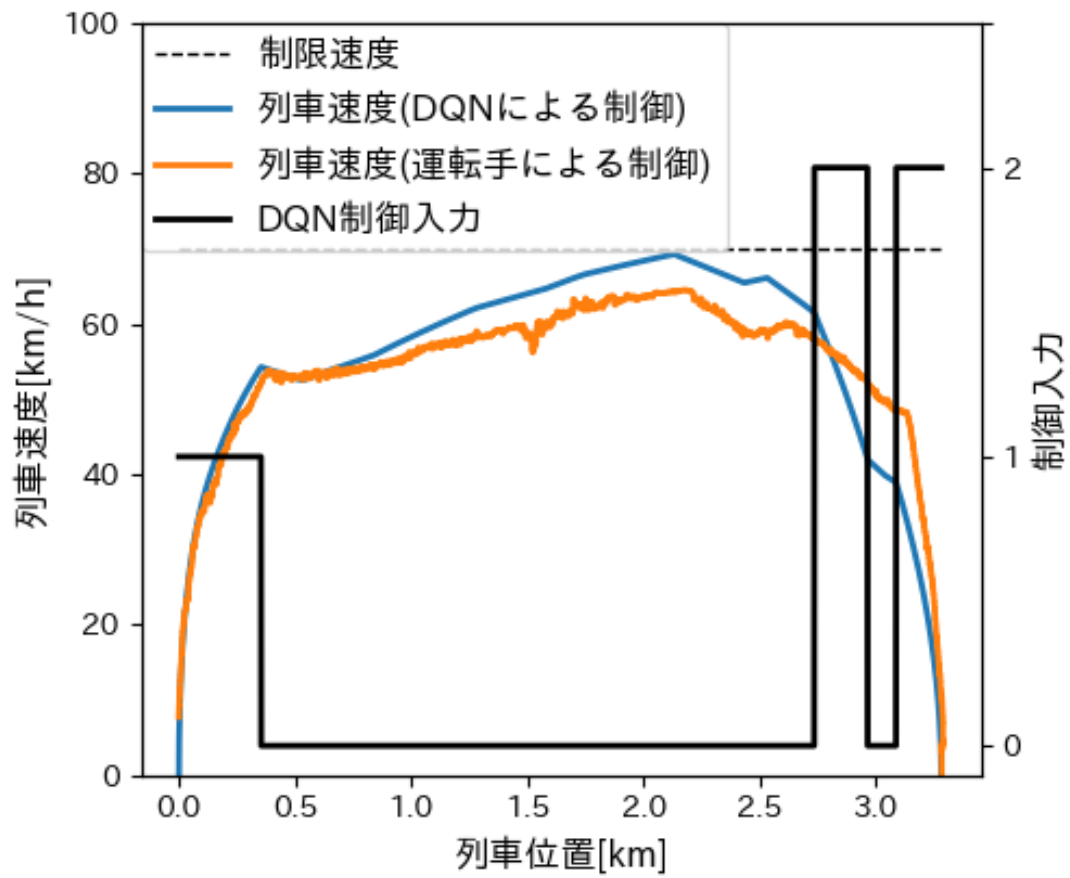


図 3.5 駅 A～駅 B 運転曲線図

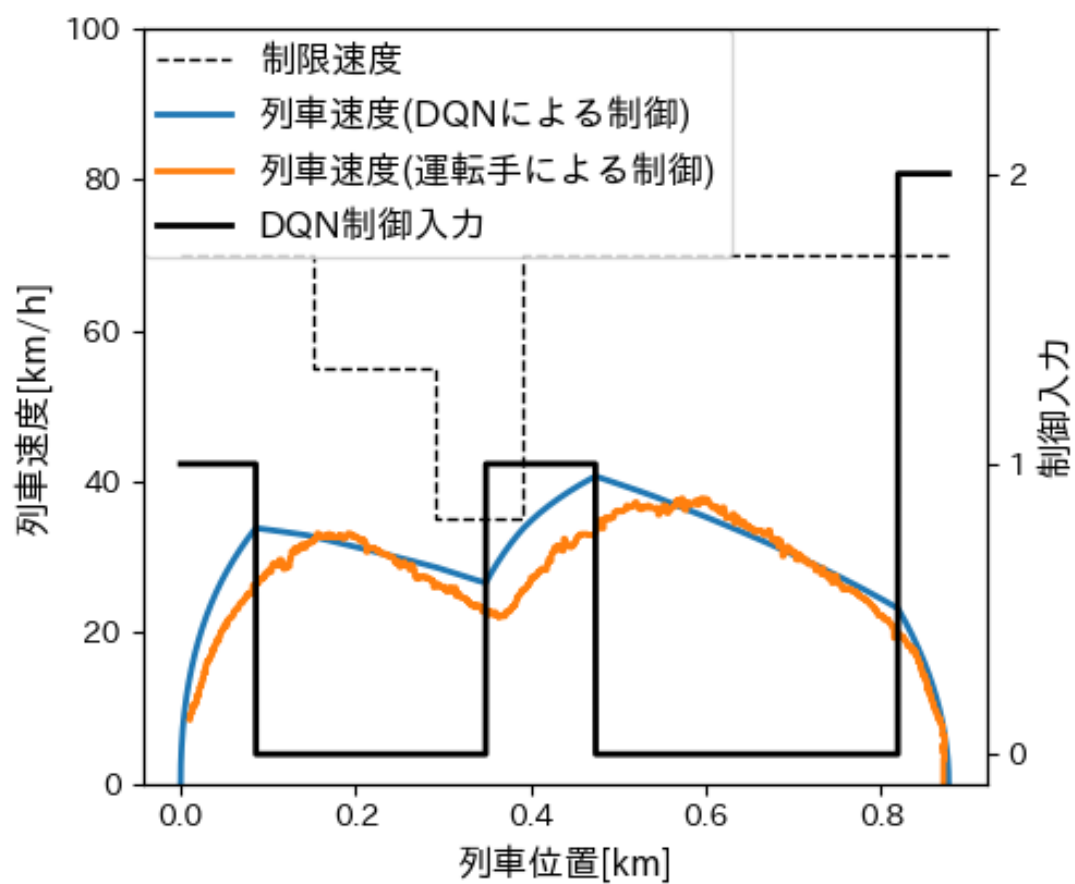


図 3.6 駅 C～駅 D 運転曲線図

表 3.2 各区間における t_e , d_e

区間	絵機関発生遅延時間[s]	駅停車位置誤差[m]
A 駅～B 駅	-0.6	0.25
C 駅～D 駅	-3.0	0.04

3.5 考察

A 駅～B 駅間は図 3.3 に示されるように区間中程に急な下り勾配が存在し、図 3.5 より DQN による制御，運転手による制御共に下り勾配を利用することで消費エネルギーを抑える制御が確認できる．また，図 3.5 の DQN 制御入力から頻繁に制御を切り替えていないため，DQN が乗り心地の面で良好な制御を行っていることが分かる．

C 駅～D 駅間は区間途中で制限速度が下がる区間であるが，図 3.6 より DQN による制御，運転手による制御共に制限速度が下がる前に加速から惰行に切り替えることで，制限速度の超過を防止する制御が確認できる．

表 3.2 より，いずれの走行区間においても駅間発生遅延時間，駅停車位置誤差が共に小さい値に収まっているため，DQN が定時運転，駅停車位置精度の面で良好な制御方策を学習していることが分かる．また，JR 山手線のホームドアが要求する停車位置精度が 0.35m であることから，学習した制御方策の停車位置精度が十分に高いと言える．

式(3.10)に示したように本章では駅到着遅延が発生した場合にペナルティを与え，早着した場合はペナルティを与えていないが，表 1 より大きな早着は発生していない．これは，走行時間に余裕がある場合に加速ではなく惰行することで，消費エネルギーを抑えることが出来るためであり，DQN が式(3.12)に示す省エネルギー性を評価する報酬を最大化することで消費エネルギーを抑える制御を学習していることが分かる．

本章では，DQN による列車制御方策の学習手法を提案し，この手法により学習したモデルが定時運転，駅停車位置精度，省エネルギー性，乗り心地などの面で良好な制御を行えることを運転曲線や駅到着遅延時間，駅停車位置誤差から確認した．また，DQN の走行による運転曲線と営業列車の走行による運転曲線を比較し，DQN が実際の運転手と概ね同様の制御を行っていることから，DQN による制御の妥当性を確認した．

本章では列車総重量を一定の値に固定した上で DQN の学習，列車運行シミュレータによる検証を行ったが，実際の環境においては乗客の乗降により列車総

重量は変化し，これにより引張力，走行抵抗，曲線抵抗による走行特性が変化する．したがって，このような環境においては列車総重量の変化を考慮した列車制御方策を学習する必要がある．次章では列車総重量が駅間走行毎に異なる環境における列車制御方策の学習手法を提案する．

第4章 列車総重量が走行毎に異なる環境 における列車制御方策の学習

4.1 章概要

前章では、走行毎に環境が変化しない理想環境における列車制御の学習手法を提案したが、実際の環境においては乗客数の変化に伴い列車総重量が変化し、走行特性に影響を与えるため、列車総重量を制御方策の入力に組み入れる、もしくは制御による状態への影響をフィードバックするといった対策が必要である。本章では、前状態における列車加速度を制御方策に入力することで、列車総重量の変化を考慮した列車制御方策の学習手法を提案する。

4.2 提案手法

第3章で提案した手法を基に、列車総重量の変化を考慮するため、列車運行シミュレータ、報酬調整定数、ネットワーク入力、ネットワーク構成、学習環境を変更した。

4.2.1 列車運行シミュレータ

3.2.1 で示した列車運行シミュレータは列車総重量パラメータを保持していない。そこで列車加速度を表す式(3.2)に列車総重量倍率 k_w を導入する。式(4.1)に列車総重量倍率 k_w を導入した列車加速度の算出式を示す。

$$\frac{dv}{dt} = \begin{cases} -\frac{\frac{R_r}{k_w} + R_g + R_c}{28.34467} & \text{if } u = 0 \\ -\frac{8.64}{k_w} - \frac{\frac{R_r}{k_w} + R_g + R_c}{28.34467} & \text{if } u = 1 \\ \frac{\frac{F - R_r}{k_w} - (R_g + R_c)}{28.34467} & \text{if } u = 2 \end{cases} \quad (4.1)$$

ここで、 $k_w = 1$ で空車時を表し、 $k_w=1.3$ で乗車率 100%時、列車総重量が空車時と比べ 1.3 倍であることを表す。

4.2.2 報酬

第 3 章では、報酬調整定数を $k_{sl}=1$, $k_{de}=0.04$, $k_{te}=7$, $k_{rc}=5$, $k_{ee}=0.02$ としていたが、本章の環境では列車総重量が重い場合に対応するため消費エネルギーを大きくしてでも定時性を維持する制御が予測される。そのため、省エネルギー性を評価する項の奉秋調整定数である k_{ee} を 0.05 とした。

4.2.3 ネットワーク

列車総重量の変化を考慮した列車制御方策を学習するため、ネットワークの入力に(直前の制御による加速度－空車時における直前の制御による加速度)を加える。ここで、直前の制御による加速度は列車運行シミュレータ上で走行した際に実際に得られた値、空車時における直前の制御による加速度は式(4.1)において $k_w = 1$ として求めた。

ネットワーク入力の変化に合わせ、新たに構成したネットワークの構成図を図 4.1 に示す。

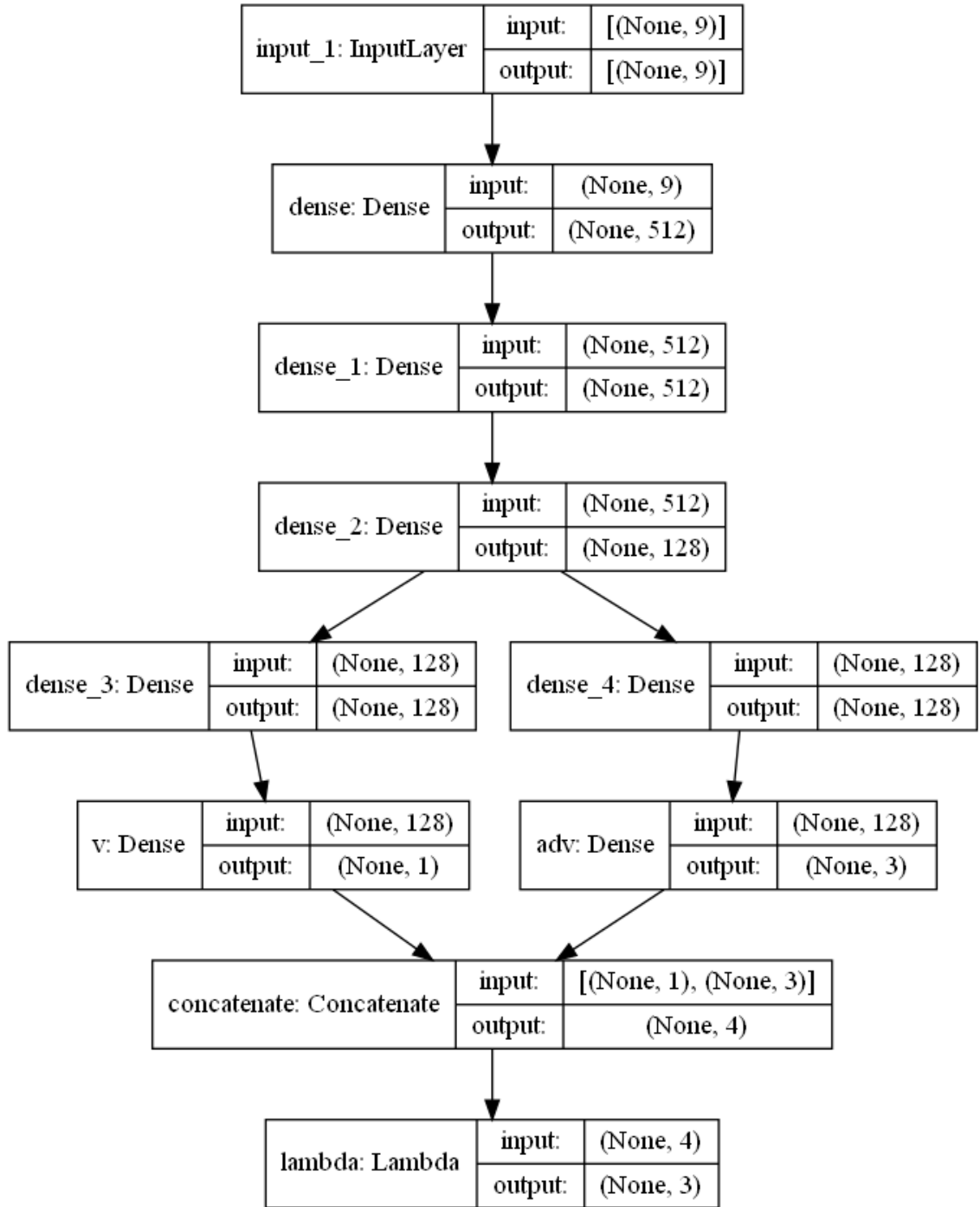


図 4.1 ネットワーク構成図

4.2.4 学習環境

列車総重量の変化を考慮した列車制御方策を学習するため、学習時において列車総重量係数 k_w を 1~1.3 の範囲でランダムに適用した。ただし、学習初期の段

階から列車総重量係数 k_w を広い範囲でランダムに適用した場合，学習が不安定になり Q 値が発散するため，初回学習時の列車総重量係数 k_w を 1 とし，列車総重量係数 k_w の上限値を 1 学習あたり 0.0001，上限となる 1.3 まで増加させた．

4.3 評価方法

学習済みの列車制御方策を用いて列車運行シミュレータ上で駅 A～駅 B 間を列車総重量 1.0 倍，1.1 倍，1.2 倍，1.3 倍において走行し，得られた運転曲線図から安全性，乗り心地を，列車位停車位置誤差，駅間発生遅延時間から定時性，省エネルギー性を評価した．

4.4 結果

列車運行シミュレータ上で学習したモデルにより A 駅～B 駅間を走行することで得た制御入力，運転曲線図を図 4.2 に示す．ここで，制御入力 0 は惰行，1 は加速，2 は減速を表す．また，図 4.2 に示した制御方策による走行における駅間発生遅延時間，駅停車位置誤差を表 4.1 に示す．

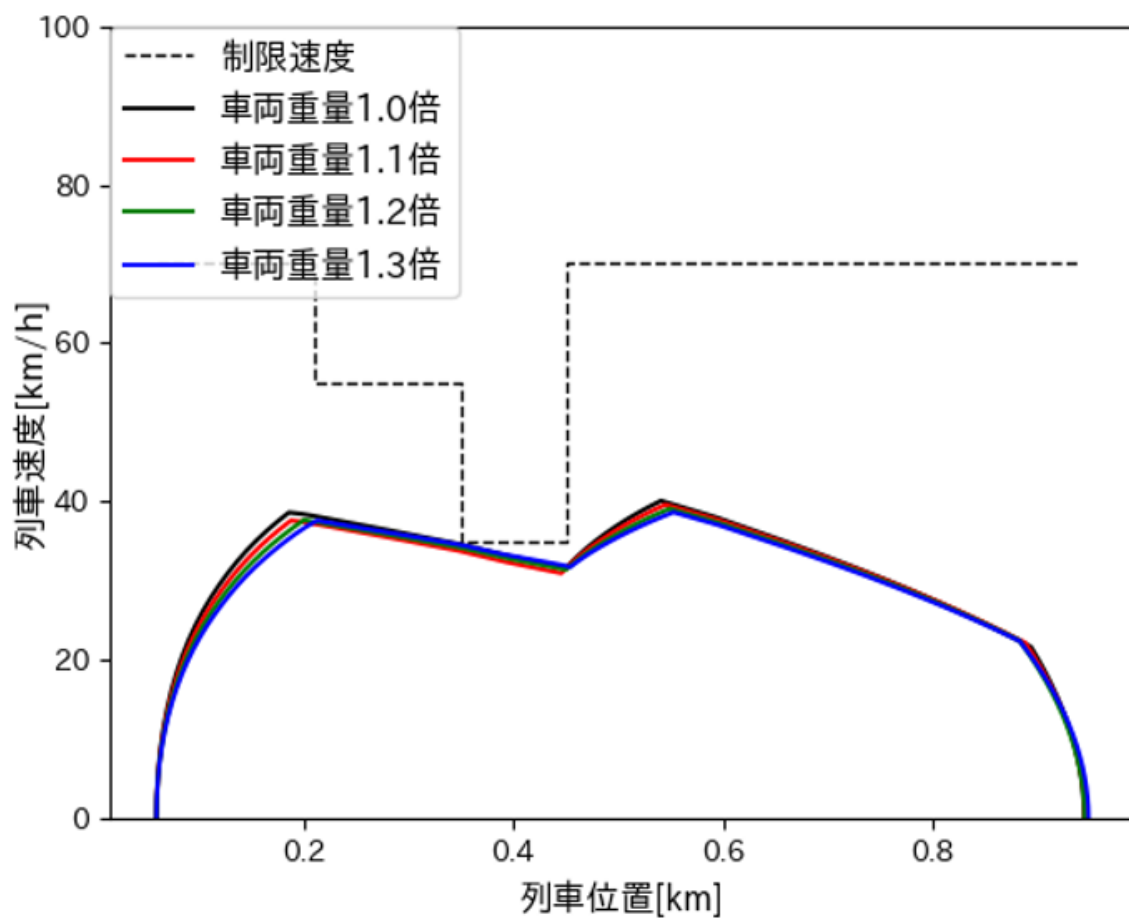


図 4.2 駅 A～駅 B 間運転曲線図

表 4.1 各列車総重量倍率における走行結果

列車総重量倍率	駅間発生遅延時間[s]	停車位置誤差[m]
1.0	114.2	0.24
1.1	116.6	0.49
1.2	118.1	0.17
1.3	120.2	0.76

4.5 考察

図 4.2 より，すべての条件において制限速度を超過しておらず，また頻繁な制御の切り替えが見られないことから，学習した制御方策が安全性，乗り心地の面で良好であると言える．また，本章では駅到着遅延が発生した場合にペナルティ

を与え、早着した場合はペナルティを与えていないが大きな早着は発生していないため、走行時間に余裕がある場合に加速ではなく惰行することで、消費エネルギーを抑える省エネルギー制御を学習していることが分かる。

表 4.2 より、大きな駅間発生遅延時間は発生していないが、列車総重量 1.3 倍において 0.2s の駅間発生遅延時間が発生していることから、学習した制御方策が定時性において改善の余地がある考えられる。また、列車総重量倍率 1.1 倍、1.3 倍において停車位置誤差が JR 山手線が求める停車位置誤差 0.35m を超過しており、厳しい停車位置誤差が求められるホームドアが導入されている路線において学習した制御方策を用いる場合、停車位置精度の面で不十分である。しかし、厳しい停車位置精度を求めるホームドアを導入している路線においては定位置停止装置(Train Automatic Stop-position Controller: TACS)が導入されており、本章で提案した手法により学習した列車制御方策と TACS を併用することで、高い停車位置精度を保証することは容易であると考えられる。

第5章 前方列車が存在する環境における 列車制御方策の学習

5.1 章概要

第3章、第4章では、前方列車が存在しない環境における列車制御の学習手法を提案したが、実際の路線においては前方列車が存在し、前方列車の停車に伴い追突を回避するため一時的に停車する場合がある。そこで本章では、前方列車が存在する環境、存在しない環境両方における DQN による列車制御方策の学習手法を提案し、これを評価する。

5.2 提案手法

第3章で提案した手法を基に、前方列車の存在を考慮するため、列車運行シミュレータ、報酬調整定数、ネットワーク入力、ネットワーク構成、学習環境を変更した。

5.2.1 報酬

前方列車が存在する場合、前方列車が存在しない場合と比べ駅間走行猶予時間が短く、また発車後より短い距離で停車することを要する場合がある。そこで報酬関数における停車位置精度を評価する関数である r_{de} を式(5.1)に変更し、乗り心地閾値 TH_{rc} を 7s、報酬調整定数を $k_{sl} = 1$, $k_{de} = 0.4$, $k_{te} = 0.5$, $k_{rc} = 0.35$, $k_{ee} = 0.001$ とした。

$$r_{de} = \begin{cases} (\max(|x_s - x|, 0.1))^{-0.5} & \text{if } x_s \leq x \wedge v = 0 \\ 0 & \text{otherwise} \end{cases} \quad (5.1)$$

ここで、 x は列車位置[m]、 x_s は規定停車位置[m]を表す。

5.2.2 ネットワーク

第 3 章, 第 4 章では列車が停車する位置は目的駅のみであったが, 前方列車が存在する環境においては駅間での停車を要する場合がある. そこで, 第 3 章のネットワークの入力に新たに停車位置までの猶予距離を加えた. ここで駅間での停車を要しない場合は停車位置 = 駅停車位置までの猶予距離とした. また停車位置まで十分に短い時間で走行する必要があるため, 第 3 章における計画駅間走行時間を式(5.2)に示す目標走行時間に置き換えた.

$$T_{target} = \frac{\int_{x_f}^{x_l} \frac{x}{v_l(x)} dx}{\int_{x_d}^{x_a} \frac{x}{v_l(x)} dx} T \quad (5.2)$$

ここで, T は計画駅間走行時間[s], x_d は出発駅位置[m], x_a は目的駅位置[m], x_f は列車位置[m], x_l は停車位置[m], $v_l(x)$:は位置 x における制限速度を表す.

ネットワーク入力の変更に合わせ新たに構成したネットワークを図 5.1 に示す.

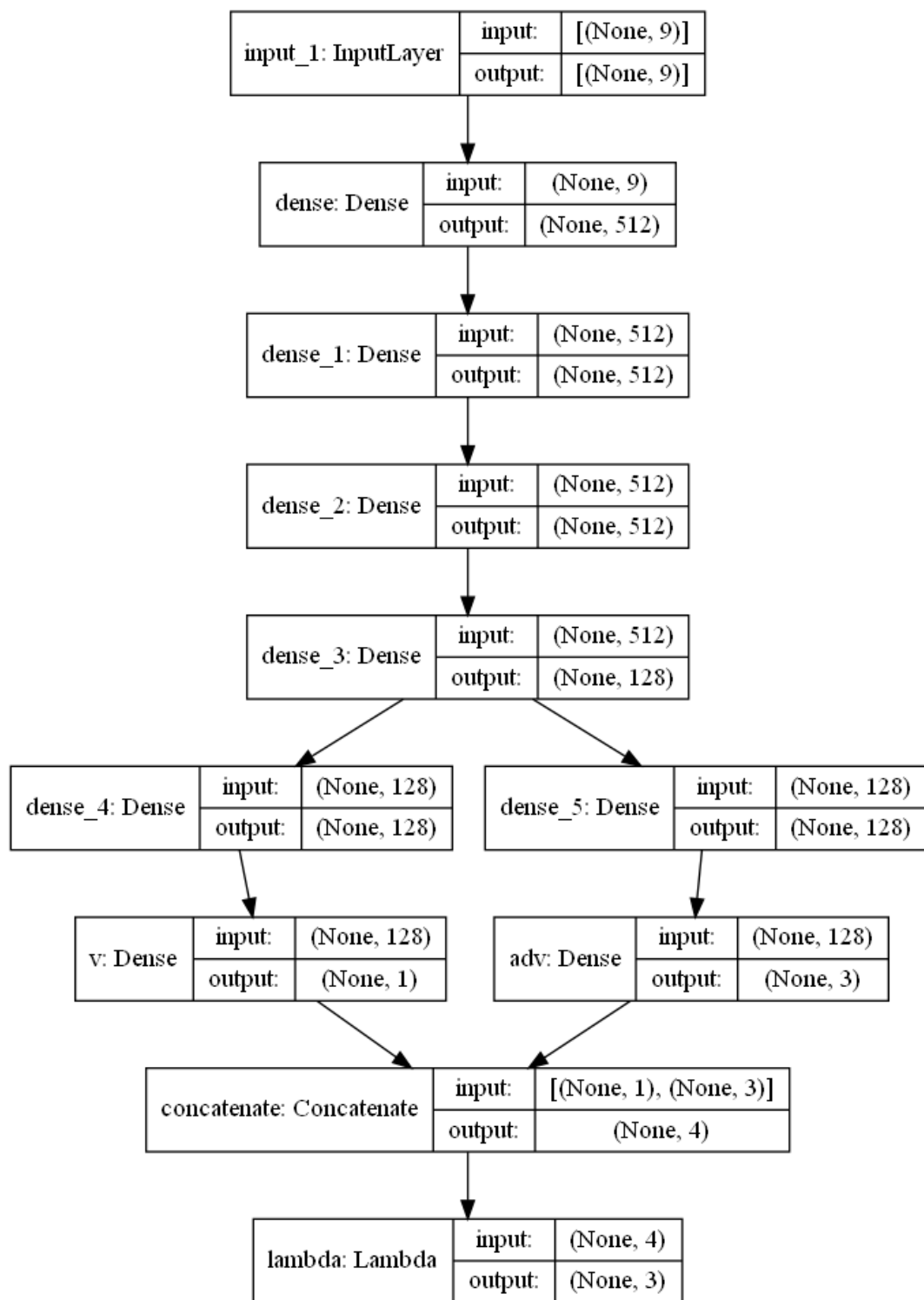


図 5.1 ネットワーク構成図

5.2.3 学習環境

前方列車が存在する場合、制御対象列車は前方列車の制御に関わらず前方列車手前で停車する必要がある。したがって、前方列車が存在する場合、停車位置は常に前方列車位置の手前となり、前方列車位置を変化させず前方列車位置手前を制御対象列車の停車位置とし、停車位置 100m 以内で停車した場合にシミュレーションを終了する環境で学習することで、前方列車への追突を回避する列車制御方策を学習することが出来る。そのため、本章では駅間に指定した任意の位置手前 100m 以内での停車を要する環境を前方列車が存在しない環境に加えて学習する。また、前方列車に追従する制御も学習するため駅間途中から発車し、目的駅で停車する環境においても学習した。

5.3 評価方法

学習済みの列車制御方策を用いて列車運行シミュレータ上で駅 E～駅 F 間を前方列車が存在しない環境、前方列車が存在し前方列車が一時的に駅間で停車する環境において走行し、得られた制御から安全性、乗り心地を、列車位停車位置誤差、駅間発生遅延時間から定時性、省エネルギー性を評価した。ここで前方列車の制御はあらかじめ手動で指定したものである。駅 E～駅 F 間の勾配を図 5.2 に示す。

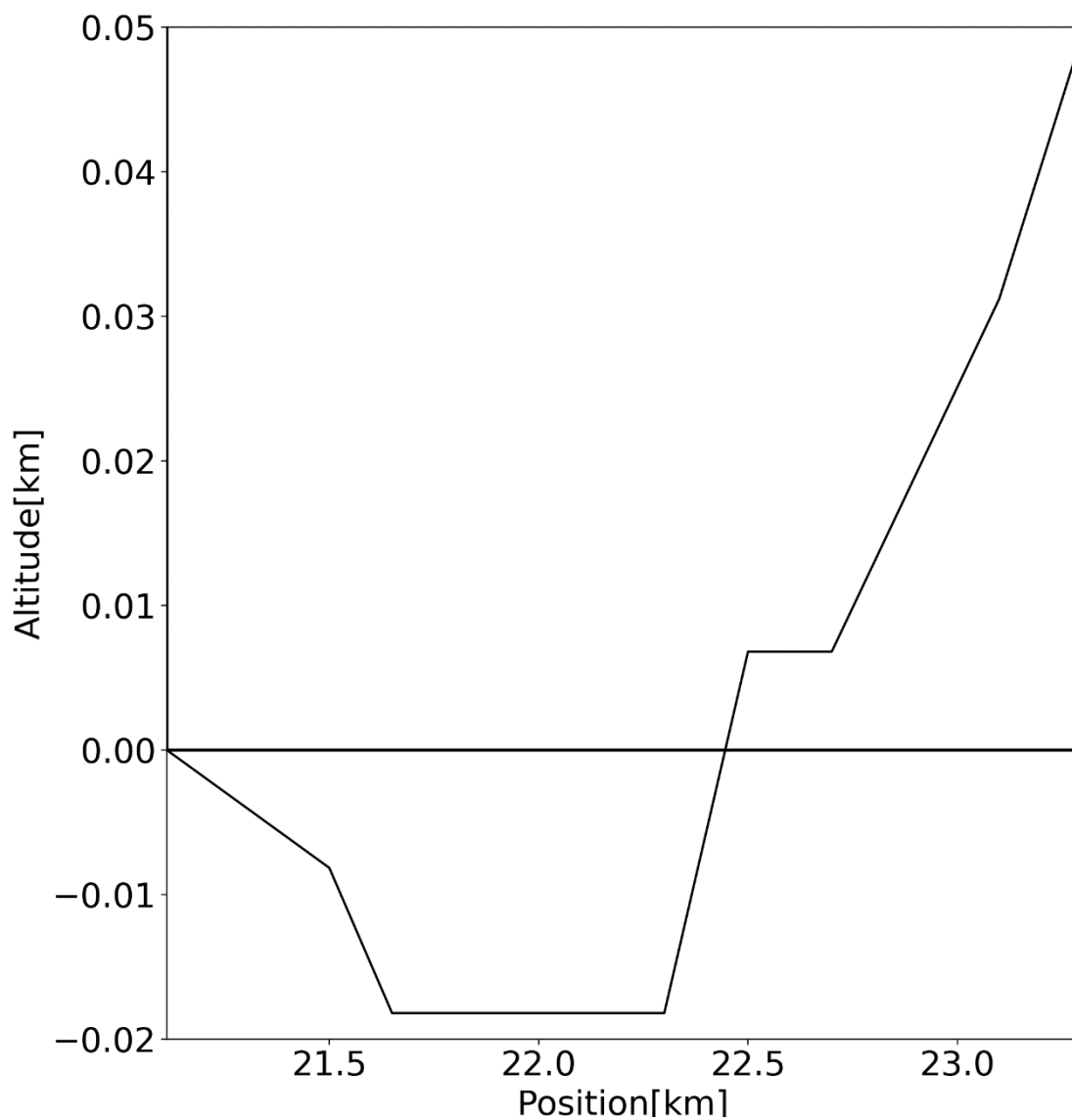


図 5.2 駅 E～駅 F 間勾配

5.4 結果

列車運行シミュレータ上で学習したモデルにより前方列車が存在しない環境において駅 E～駅 F 間を走行した運転曲線図を図 5.3 に、同制御による各時刻における速度, 制御入力を図 5.4 に示す. この走行において, 停車位置誤差は 0.21m, 駅間発生遅延時間は-7.5s であった.

また, 前方列車が存在し前方列車が一時的に駅間で停車する環境において駅 E

～駅 F 間を走行し得られた各時刻における前方列車及び後方列車の位置，速度を図 5.5 に示す．この走行において駅停車位置誤差は 0.16m であった．

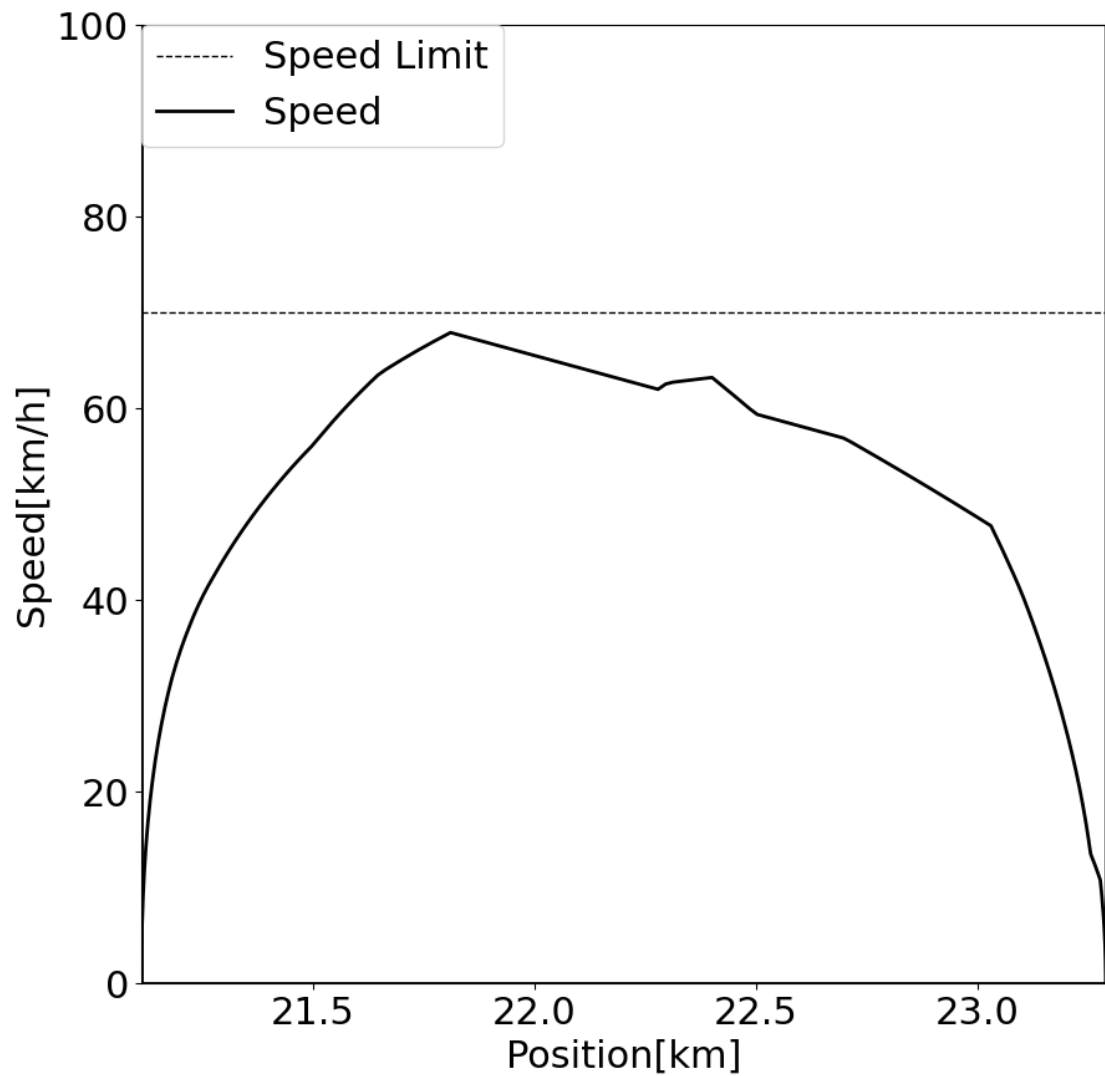


図 5.3 前方列車が存在しない環境における運転曲線図

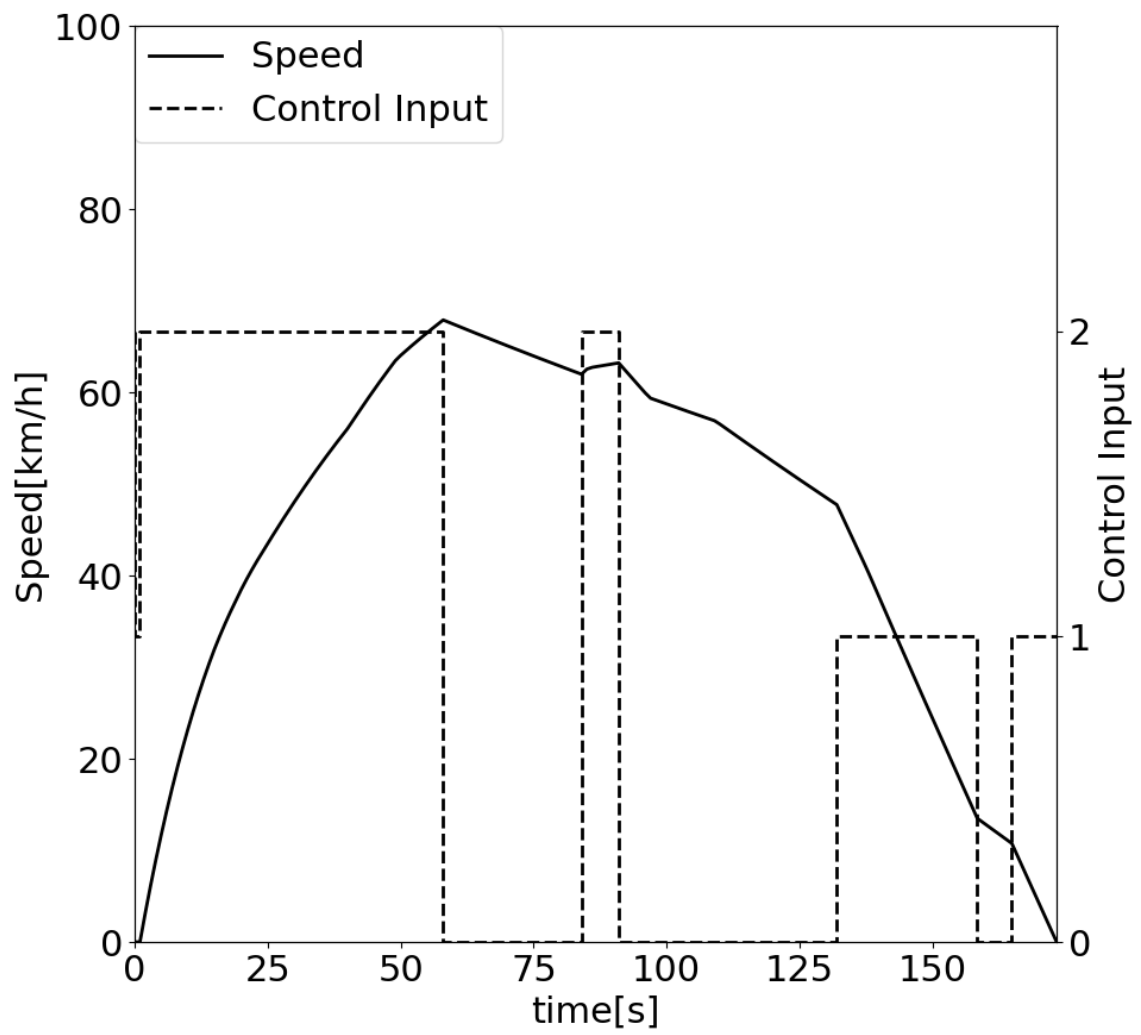


図 5.3 前方列車が存在しない環境における速度，制御入力

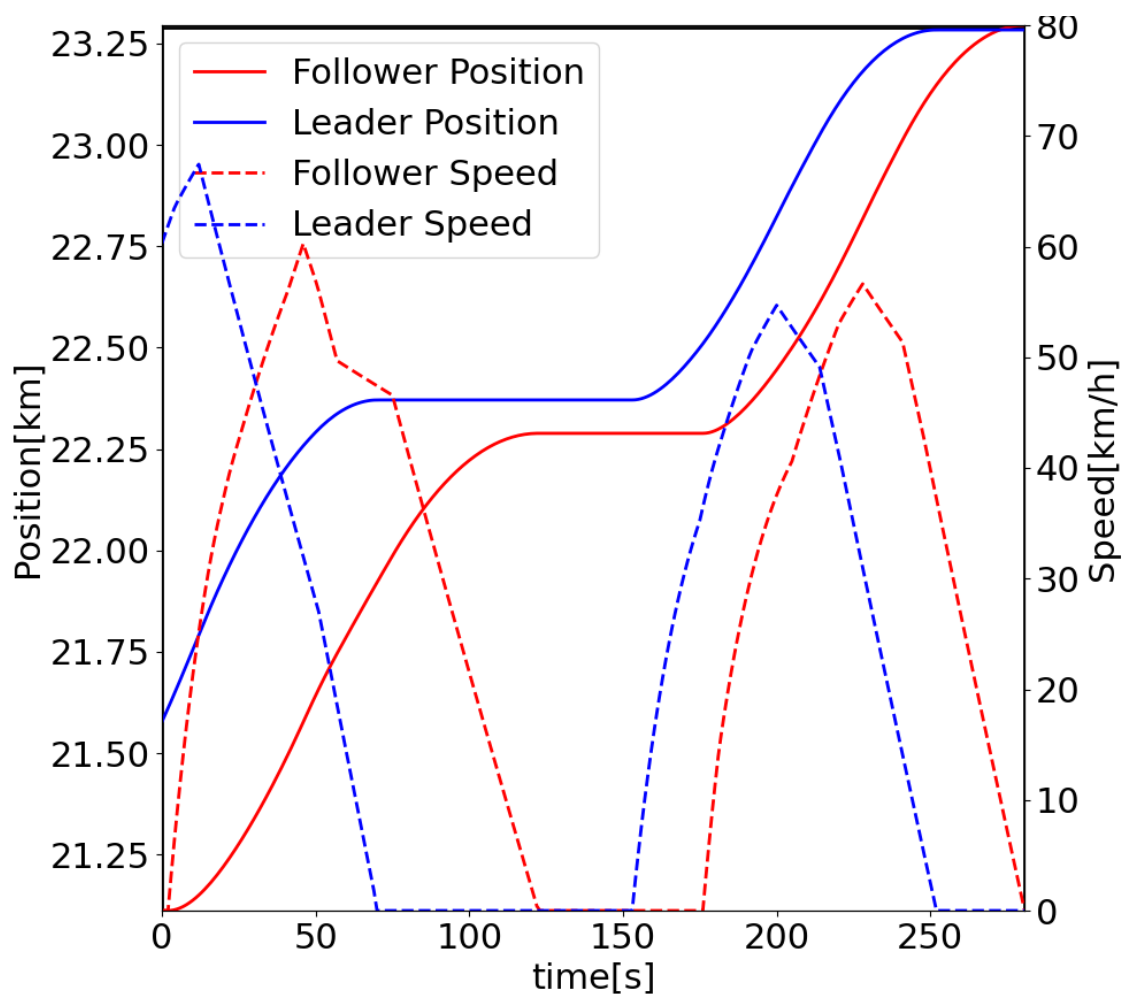


図 5.4 前方列車が存在する環境における前方列車，制御対象列車の位置，速度

5.5 考察

図 5.2 より，制限速度を超過しておらず，図 5.3 より頻繁な制御の切り替えを行っていないため，学習した制御方策が安全性，乗り心地の面で良好であると言える．駅停車位置誤差十分に小さい値であることから停車位置精度の面で良好な制御であると言え，また本章では駅到着遅延が発生した場合にペナルティを与え，早着した場合はペナルティを与えていないが大きな早着は発生していないため，走行時間に余裕がある場合に加速ではなく惰行することで，消費エネルギーを抑える省エネルギー制御を学習していることが分かる．

図 5.4 より，前方列車が停車した場合に一時停車し，前方列車の発車後に再び

発車していることから、追突を回避する制御、前方列車に追従する制御を学習していることが確認できる。また、制御対象列車の駅停車位置誤差が十分に小さい値であることから、学習した制御方策が停車位置精度の面で良好であると言える。

第6章 結論

本研究では、理想環境、列車総重量が走行毎に異なる環境、前方列車が存在する環境における DQN による列車制御方策の学習手法を提案し、これを評価した。

理想環境における制御方策が、実際の運転手と同様の制御を行うこと、安全性、定時性、駅停車位置精度、乗り心地、省エネルギー性の面で良好な制御を行うことを確認した。

次に、列車総重量が走行毎に異なる環境における制御方策が、安全性、乗り心地、省エネルギー性の面で良好な制御を行うことを確認した。しかし特定条件において 0.2s 程度の駅間発生遅延時間が発生するため、改善の余地がある。また、一部条件において 35cm 以上の駅停車位置誤差が発生するため、高い停車位置精度が求められる環境においては TACS の併用が必要となる。

また、前方列車が存在する環境における列車制御方策が、前方列車が存在しない環境、前方列車が存在し前方列車が駅間で一時停車する環境両方において安全性、定時性、駅停車位置精度、乗り心地、省エネルギー性の面で良好な制御を行うことを確認した。

本論文で提案した列車制御方策は NN の出力を基に決定するものであるため、安全性を保証することは困難である。したがって、安全性の確保のため ATC や ATS 下での使用が望まれる。また、高い駅停車位置精度を要する路線においては TACS の併用が望まれる。

本論文では前方列車が存在する環境において、前方列車に追突しない程度に接近する制御を学習したが、実際の路線においては分散乗車のため前方列車および後方列車との間隔を適切に調整する必要があり、今後はこの様な制御方策の学習手法に関して検討が望まれる。

謝辞

本研究は、株式会社京三製作所の協力の下行う事が出来ました。ご指導いただいた関係者各位に篤く御礼申し上げます。また、常日頃指導をいただいた高橋聖教授、中村英夫教授、福田卓海助手に篤く御礼申し上げます。最後に、研究環境整備のため助力をいただいた高橋研究室の皆様に篤く御礼申し上げます。

参考文献

- [1] 福井鉄道株式会社, “福武線ダイヤ改定の実施について,” 2023.
- [2] 畝. 達夫, “強化学習,” 人工知能学会誌, 9 卷 6 号, p.830-836, 1994.
- [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kum, “Human-level control through deep reinforcement learning,” *Nature*, vol.518, pp.529-533, 2015.
- [4] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver, “Prioritized Experience Replay,” *ICRL*, 2016.
- [5] Dan Horgan, John Quan, David Budden, Gabriel Barth-Maron, Matteo Hessel, Hado van Hasselt, David Silver, “Distributed Prioritized Experience Replay,” *ICLR*, 2018.
- [6] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, Nando de Freitas, “Dueling Network Architectures for Deep Reinforcement Learning,” *ICML*, 2016.
- [7] Hado Van Hasselt, Arthur Guez, and David Silver, “Deep reinforcement learning with double q-learning,” *AAAI*, vol 2, pp. 5, 2016.